

# How Recruitr finds candidates, and how it scores them

Two halves of one system. **Sourcing** turns a job description into a LinkedIn search that returns real people. **Matching** scores those people against what the job actually requires, and shows its evidence. This document covers the architecture, the workflow, every prompt, and the exact scoring maths.

Scoring version match-scoring-4      Search actor harvestapi/linkedin-profile-search      Vector store mongo

OVERVIEW

- The big picture
- The Role Spec spine

PART 1 • SOURCING

- Workflow
- The Search Brief
- The Strategist
- Filters & enums
- The Broadener
- The domain guard
- The pre-screen gate
- Enrichment

PART 2 • MATCHING

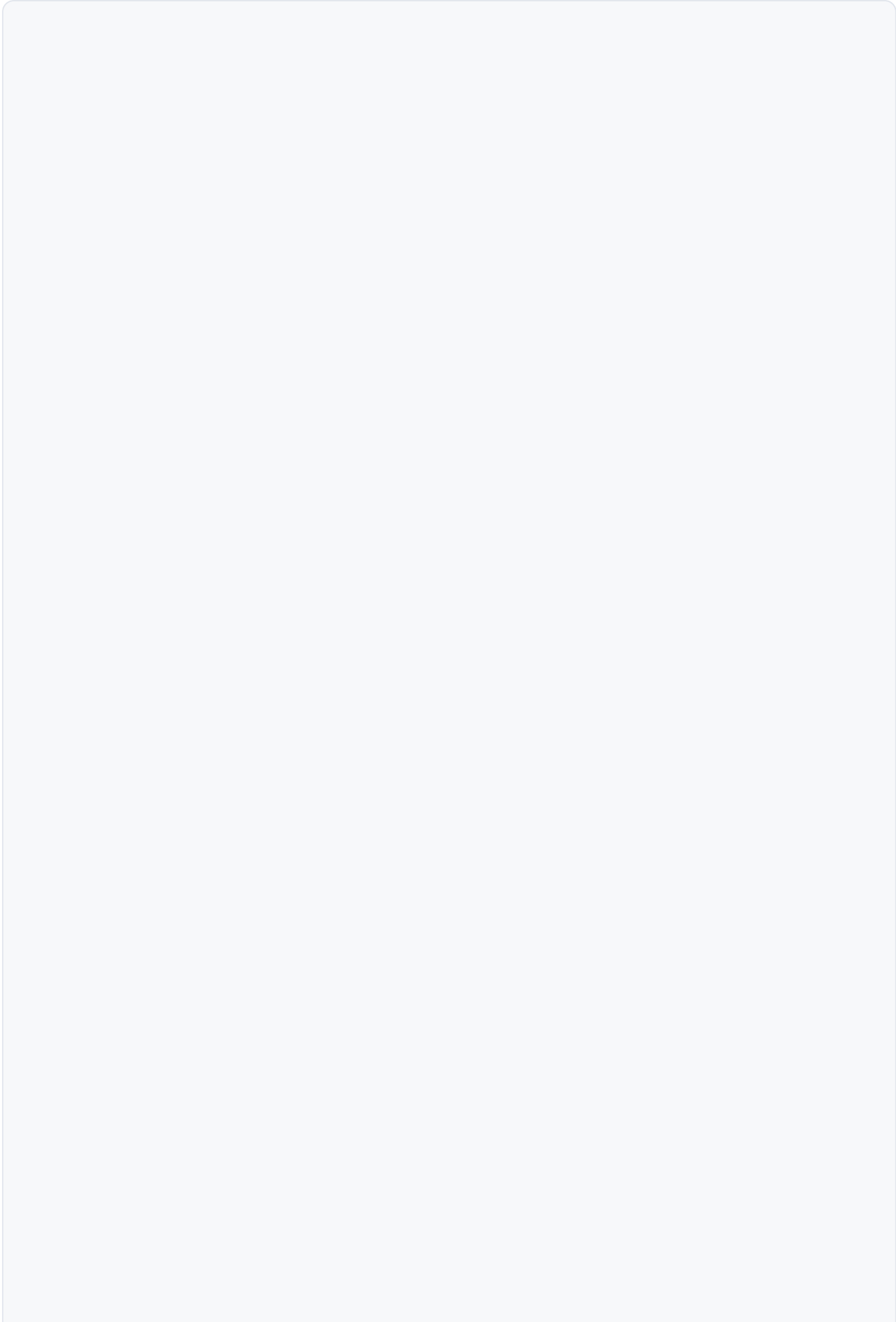
- Workflow
- Weights & redistribution
- Skill matching rules
- The evidence pool
- The must-have ceiling
- A worked example
- Gaps vs partials

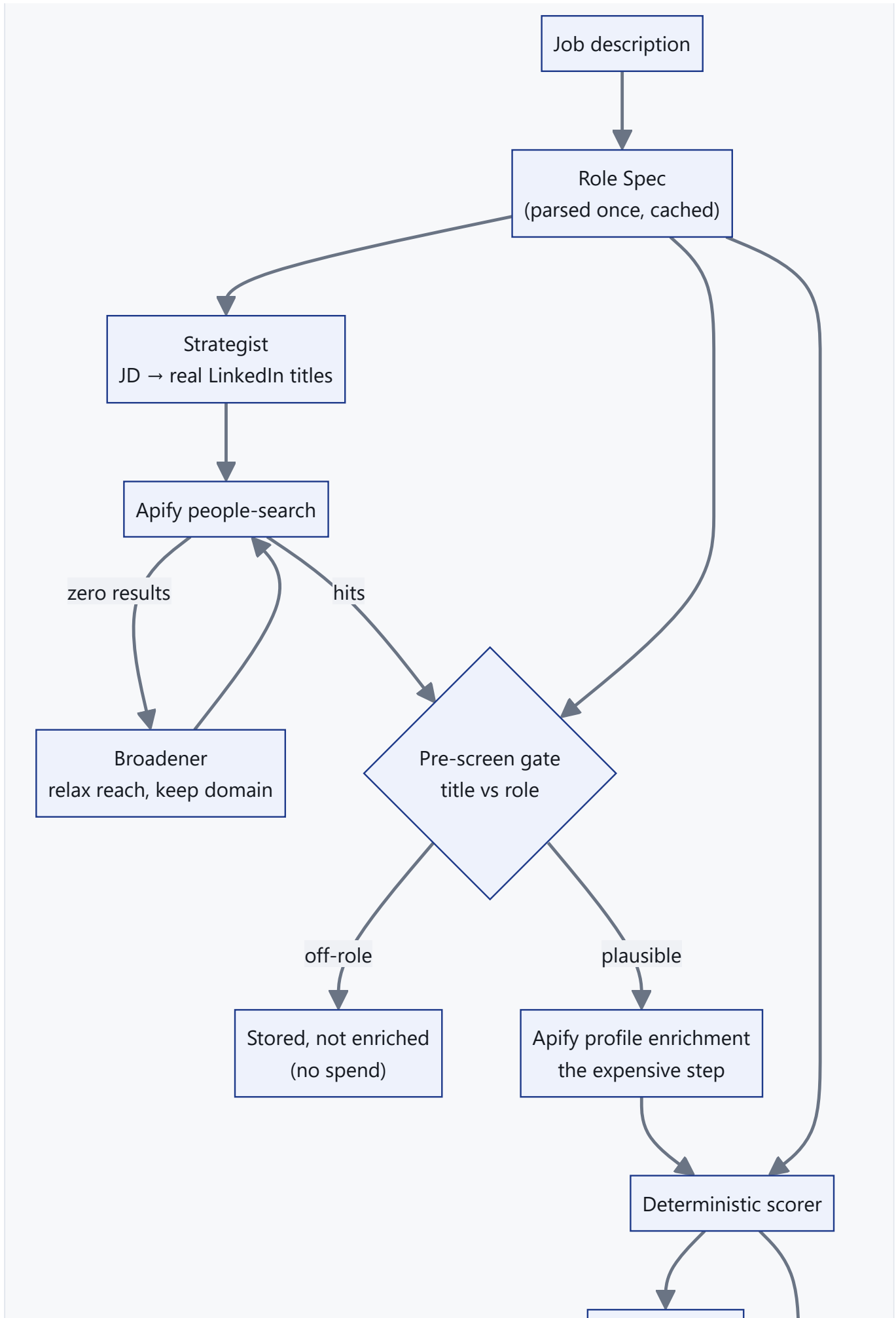
REFERENCE

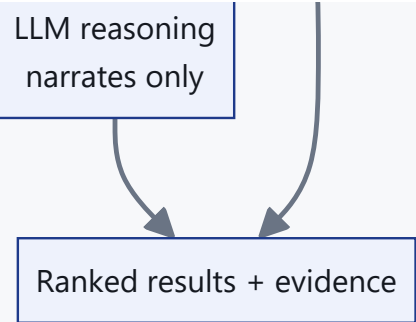
- All prompts
- Config & cost
- Known traps

# The big picture

The system's organising idea: **the requirement drives everything, and it is computed once.** A job description is parsed into a structured Role Spec, and that same object aims the LinkedIn search, gates the spend, and grades the results. Sourcing and matching never form separate opinions about what the job needs.







```
graph TD; A[LLM reasoning narrates only] --> B[Ranked results + evidence];
```

LLM reasoning  
narrates only

Ranked results + evidence

Two properties are worth calling out, because they are what make the numbers trustworthy:

- **The gate sits before the money.** A LinkedIn search returns a title and location for free; deep enrichment costs per profile. Anyone whose title has nothing to do with the role is dropped before we pay for them.
- **The LLM never states a fact.** Every score, every gap, every skill credit is computed in code. The model only writes the prose. It cannot contradict the evidence, because it is not the source of it.

# The Role Spec spine

`role_spec_service.py` owns one canonical structured requirement per job, stored as a `parsed_jds` document, keyed by a hash of the JD text and linked to the job by `sourceJobId` .

ROLE SPEC — SHAPE

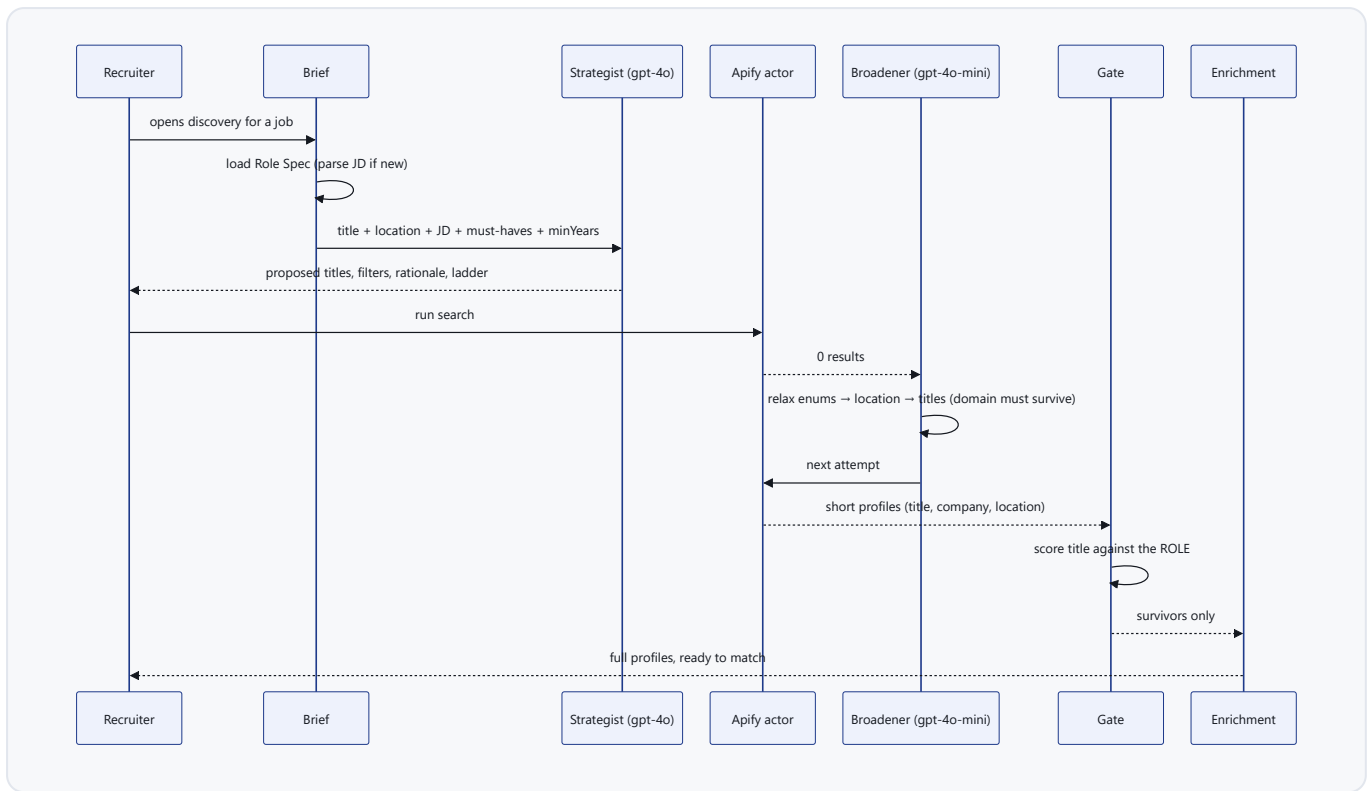
parsed\_jds

```
{
  "sourceJobId": "6a58c9b76753c3ece2f47999",
  "textHash": "<sha256 of the JD text>",
  "rawText": "<the job description>",
  "requirements": {
    "title": "Sachbearbeiter Entgeltabrechnung",
    "mustHaveSkills": ["SAP HR3", "Entgeltabrechnung", "Arbeitsrecht",
                      "Sozialversicherungsrecht", "Lohnsteuerrecht"],
    "niceToHaveSkills": ["Prozessoptimierungen"],
    "minYears": 3,
    "location": null,
    "seniority": "Senior",
    "responsibilities": [...]
  },
  "embedding": { "model": "text-embedding-3-small", "dim": 1536, "vector": [...] }
}
```

What the hash buys: **one LLM parse and one embedding per distinct JD**. Re-running a match, or opening the discovery form twice, reuses the parse. Editing the JD changes the hash, so a stale spec is never silently reused.

# Part 1 · Candidate sourcing

The problem this half exists to solve: **a job posting title is employer language; a LinkedIn headline is how a person describes themselves.** They are frequently different words for the same job, and searching the posting title verbatim is the single biggest cause of a search returning nobody.



## The Search Brief

Assembled by `sourcing/brief.py` from what the database already knows — the recruiter never retypes it. The **must-have skills come from the Role Spec**, which is what lets the Strategist aim at the requirement rather than the job title alone.

```
jobTitle:      "Sachbearbeiter Entgeltabrechnung (SAP HR3)"    # required
jobLocation:   "Frankfurt, Germany"
companyName:   "Payroll Test Co"
companyIndustry: ""
jobDescription: "<raw JD – strongest signal for vocabulary and language>"

# — from the Role Spec (the requirement) —
mustHaveSkills: ["SAP HR3", "Entgeltabrechnung", "Arbeitsrecht", ...]
niceToHaveSkills: ["Prozessoptimierungen"]
minYears:      3.0
seniorityHint: "Senior"

# — optional recruiter overrides —
targetCompanies / excludeCompanies / languages / workModel / notes
```

## The Strategist

One reasoning call, no tools, run once when the discovery form opens. It translates the role into titles real people carry, and pre-plans a broadening ladder. Model:

`openai:gpt-4o` . If no LLM key is configured it degrades to a literal title prefill with `confidence: 0` — it is an enhancement, never a hard dependency.

You are a technical sourcing strategist. You turn a job opening into a LinkedIn people-search that returns REAL candidates.

The single most important thing you understand: a job POSTING title is written in employer/HR language, but you are searching PROFILE titles, which are written in the member's own words. They are frequently different, and searching the posting title verbatim is the #1 cause of a zero-result search.

Examples of the translation you must perform:

- "SAP Consultant FI" → nobody's headline says that. Real titles: "SAP FICO Consultant", "SAP FI Consultant", "SAP Finance Consultant", "SAP FI/CO Consultant", "Senior SAP Consultant". FI is almost always paired with CO in practice – include the FICO variants.
- "Java Developer II" → the "II" is an internal grade, never a headline. Use "Java Developer", "Software Engineer", "Backend Developer".
- "Growth Marketing Ninja" → a real title is "Growth Marketing Manager", ...
- "Cloud Architect (AWS)" → "Cloud Architect", "AWS Architect", ...

Rules for the filters you produce:

1. ``currentJobTitles`` – 4 to 8 titles people ACTUALLY carry on LinkedIn. Include the common abbreviation AND the expanded form when both are in use (SAP FICO / SAP Finance). Order strongest-match first. Never include internal grades (II, L3, Band 4), employment types (Contract, Permanent), or req codes.
2. ``searchQuery`` – a SHORT fuzzy keyword phrase (2-4 words), NOT the full title.
3. ``locations`` – a LinkedIn-recognisable place. Prefer the metro/city the job is in; use the country when the role is remote or the city is tiny.
4. ``seniorityLevel`` / ``function`` / ``yearsOfExperience`` – infer from the JD, not the title alone. LEAVE A FILTER NULL when the JD doesn't support it – every filter you add shrinks the result set, and a wrong filter is far worse than a missing one.
5. Skills are NOT a filter on this actor, but ``mustHaveSkills`` in the brief is the EXACT list the candidate will later be scored against – a candidate who carries none of them cannot pass, however good the title looks. So the must-haves are your primary aiming input, not a footnote: encode them in the titles and the query (must-have "S/4HANA" → add "SAP S/4HANA Consultant" as a title; must-have "Entgeltabrechnung" → "Entgeltabrechner", "Payroll Specialist", "Personalsachbearbeiter Entgeltabrechnung"). Never silently drop the signal, and never propose titles whose holders would plainly have none of the must-haves.
6. If the recruiter named target companies, put them in ``currentCompanies``.
7. LANGUAGE. People describe themselves in the language they work in. If the JD or its must-have skills are in a language other than English (German "Entgeltabrechnung", French "Chargé de recrutement"), the people you want have native-language headlines, and English titles will not find them. Emit titles in THAT language – add the English variant only where it is genuinely also in use locally (e.g. "Payroll Specialist" is common in Germany; "Labour Law Clerk" is not). Set ``profileLanguages`` to that language when the role is plainly local. Domain vocabulary is the strongest language signal – trust it over the posting title.

Enum filters MUST use one of these codes (emit the CODE, not the label):

<generated live from the actor's own ENUM\_TABLES>

Also produce:

- ``interpretedRole`` – one line naming what this job really is, in plain terms.
- ``titleReasoning`` – why the posting title does or doesn't work as a search term.
- ``rationale`` – one short entry per non-empty filter, saying why.
- ``broadeningLadder`` – 3 fallback attempts, tried in order ONLY if the search returns zero. Each step carries a COMPLETE filter set, strictly broader than

the one before.

- `confidence` – 0..1. Be honest: a vague one-line JD with no location is 0.3.
- `warnings` – anything the recruiter should know.

Be decisive and specific. Prefer fewer, higher-signal filters.

## What it produced for the German payroll role

```
interpretedRole : Entgeltabrechner with expertise in SAP HR3 and payroll laws
titleReasoning  : The job title 'Sachbearbeiter Entgeltabrechnung (SAP HR3)' is
                  specific to German payroll processing using SAP HR3, which won't
                  directly match LinkedIn profiles written in English. However,
                  titles like 'Entgeltabrechner' or 'Payroll Specialist' are
                  commonly used by professionals in this field, especially in Germany.
searchQuery      : "Entgeltabrechnung SAP HR3"
currentJobTitles: ["Entgeltabrechner", "Sachbearbeiter Entgeltabrechnung",
                  "Payroll Specialist", "Payroll Officer",
                  "Personalsachbearbeiter Entgeltabrechnung"]
locations        : ["Frankfurt, Germany"]
confidence       : 0.8
```

### WHY THIS MATTERS

Before the must-haves reached the brief, the same JD produced an English-titled search and found data engineers. The German domain vocabulary in `mustHaveSkills` is what tells the model the people it wants write *Entgeltabrechner*, not *Payroll Clerk*.

## Filters and the enum vocabulary

The Strategist emits a `SearchFilters` object whose field names match the actor's input exactly, so an agent cannot invent a filter the actor would reject. Enum values are validated against the actor's own tables and **an unrecognised value is dropped rather than passed through** — the actor fails an entire run on a bad enum instead of ignoring it.

| FILTER            | TYPE       | NOTES                                                  |
|-------------------|------------|--------------------------------------------------------|
| searchQuery       | fuzzy text | 2–4 words. Broad keyword match, ANDed with the rest.   |
| currentJobTitles  | string[]   | OR-matched. The main aiming device. Capped at 10.      |
| locations         | string[]   | Must be a place LinkedIn recognises — "Europe" is not. |
| yearsOfExperience | enum code  | 1=<1yr · 2=1–2 · 3=3–5 · 4=6–10 · 5=10+                |
| seniorityLevel    | enum code  | 120=Senior, 210=Experienced Manager, ...               |
| function          | enum code  | 1=Accounting · 10=Finance · 12=Human Resources · ...   |
| profileLanguages  | string[]   | <b>BROKEN</b> silently dropped — see Known traps.      |
| currentCompanies  | string[]   | A deliberate poach. Respected exactly.                 |

#### IMPORTANT

**Skills are not a filter on this actor.** The only way a must-have skill can influence the search is by being encoded into the titles or the query. That is why rule 5 of the Strategist prompt exists, and why the must-haves must reach the brief.

## The Broadener

Called *only* when a search returns zero. It sees the full attempt history — every filter set tried and what each returned — and proposes the next, strictly broader attempt.

Model: `openai:gpt-4o-mini` . Capped at `SOURCING_MAX_BROADEN_ATTEMPTS = 3` .

An agent rather than just walking the pre-planned ladder, because the ladder is written blind. Once you know *which* search died, the reason matters: a dead title needs different relaxation than a dead location.

You are a sourcing strategist recovering from a FAILED LinkedIn people-search: every attempt so far returned ZERO candidates.

Given the job brief and the full history of what was tried, produce the NEXT search attempt. It must be STRICTLY BROADER than every attempt already made – repeating or narrowing is a wasted paid API call.

THE ONE RULE YOU MUST NOT BREAK: widening means casting a wider net for THIS job. It NEVER means searching for a different job. The role's domain vocabulary – the words that make this role this role ("Entgeltabrechnung", "Payroll", "FICO", "S/4HANA") – must survive every attempt. Relaxing "Entgeltabrechner" to "SAP Consultant" or "HR Specialist" is NOT broadening: payroll clerks and SAP consultants are different professions, and that search returns a pile of people who cannot do the job. Zero results means "widen the net", never "change the target". If the only way you can widen further is by abandoning the domain, the answer is `give\_up`, not a search for someone else.

Relax in THIS order – cheapest, most speculative filters FIRST, and touch the titles LAST:

1. Enum filters (`seniorityLevel`, `yearsOfExperience`, `function`, `companyHeadcount`). ALWAYS start here. They are INFERRED from the JD, not stated by the member, and profiles routinely leave the underlying data blank – so they silently exclude people who DO qualify. Several set at once is the single most common cause of a zero-result search.
2. `currentCompanies` – restricts to a handful of employers.
3. `locations` – widen the city to the metro, then to the country.
4. `profileLanguages` – only if the role plausibly exists in English too.
5. TITLES, and only once 1-4 are exhausted. Generalise WITHIN the domain and keep the domain word: "Sachbearbeiter Entgeltabrechnung" → "Entgeltabrechner" → "Payroll Specialist" → "Payroll" (all still payroll). Never climb out to a generic title ("Consultant", "Specialist", "Manager") – those match everyone and nobody.

Widen ONE dimension per attempt where possible, so the result stays attributable.

Choose `decision: "broaden"` whenever ANY filter is still left to relax.

Choose `decision: "give\_up"` ONLY when the last attempt was already a bare title plus a country with no enum filters and STILL returned zero.

`reasoning` is shown to the recruiter: one plain sentence on what you changed and why, referencing the actual values. No jargon.

#### DESIGN NOTE

`decision` is an action enum ( `broaden` / `give_up` ) rather than a boolean on purpose. A negated flag ( `shouldRetry: false` ) gets inverted by the model, which then returns a perfectly good filter set flagged to be thrown away. Naming both outcomes removes the polarity it has to infer.

## The domain guard

The prompt above is **not trusted on its own**. Observed live: told the titles were too specific, the model relaxed a German payroll clerk into `SAP Consultant` / `HR`

`Specialist` and returned twelve SAP *Finance* consultants. So the rule is also enforced in code.

```
anchor = non-generic tokens of the FIRST attempt's titles
        (the Strategist's read of the job, before any relaxation)

        "Entgeltabrechner", "Payroll Specialist",
        "Personalsachbearbeiter Entgeltabrechnung", ...
        ↓ strip generic role words
        {entgeltabrechnung, payroll, personalsachbearbeiter, lohnabrechnung, ...}

for each proposed title:
    keep it only if some anchor word (or an obvious inflection) appears in it

    "Payroll Consultant" → payroll ✓ KEEP
    "SAP Consultant"    → -      X DROP
    "HR Specialist"     → -      X DROP ("specialist" is a generic role word)

if nothing survives → fall back to the Strategist's planned ladder
```

Generic role words — `consultant` , `berater` , `specialist` , `manager` , `sachbearbeiter` , `analyst` , ... — are stripped from the anchor because every profession shares them. Without that, *Payroll Specialist* and *HR Specialist* look related.

RESULT

After the guard, the same search relaxed `seniorityLevel` , `function` and the location, kept every payroll title, and returned twelve genuine *Personalsachbearbeiter Entgeltabrechnung*.

## The pre-screen gate

A search hit arrives with a title, company and location, for free. Turning it into something scoreable costs an Apify profile scrape each. The gate spends the free signal first.

**It is deliberately lopsided.** A false drop is unrecoverable — that person is never enriched, never scored, never seen again. A false keep costs one scrape and is caught by the real matcher a minute later. So it only rejects what it is confident about, and answers "*is this person plausibly in the right job family?*" — never "*do they meet the requirements*", because a title cannot evidence a skill list.

| SIGNAL                                                    | WEIGHT                                                 |
|-----------------------------------------------------------|--------------------------------------------------------|
| Title vs the titles the search <i>originally</i> aimed at | Strongest — the Strategist already translated the role |
| A must-have skill named in the title                      | Full credit — direct domain evidence                   |
| Title vs the role's own title                             | Fallback when the aim is unusable                      |
| No title at all                                           | kept                                                   |
| No Role Spec to judge against                             | kept                                                   |

#### CRITICAL

The gate screens against the **role**, never against `used_filters`. Screening against the Broadener's relaxed titles let broadening silently lower the bar: the drifted SAP-Finance results scored **100/100** and 10 of 12 were kept. Anchored to the role, 2 of 12 survive.

## Calibration, on 39 real candidates

Threshold `PRESCREEN_MIN_SCORE = 25` was set against the people actually sourced for a live SAP Consultant job — not invented:

| TITLE                                   | SCORE | VERDICT |
|-----------------------------------------|-------|---------|
| SAP Inhouse Consultant                  | 100   | KEEP    |
| Associate Consultant SAP FI/CO          | 100   | KEEP    |
| Senior Consultant (no SAP)              | 50    | KEEP    |
| CEO · Geschäftsführer · Interim Manager | 0     | DROP    |

It keeps every genuine SAP consultant, drops only executive slop, and drops ~93% of those same people when aimed at an unrelated payroll role. Dropped candidates are **stored** with their verdict and `isAccepted: false` — visible and recoverable. Only the spend is withheld.

## Enrichment

Survivors are deep-scraped via Apify into `apifyEnrichment.profile`: full name, headline, location, current title/company, `totalYears`, skills, titles, experience (with per-role skills and dates), education, certifications — plus the LinkedIn profile photo, in four sizes.

#### PHOTOS

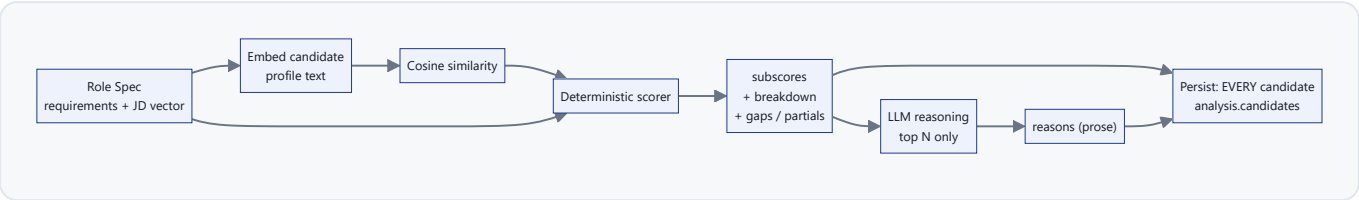
The CDN URLs are public — no cookie or referer needed — but **signed and expiring** (a few weeks). Every size is signed separately, so a 200px variant cannot be derived from the 800px URL by rewriting the path. The UI falls back to initials when a link dies.

# Part 2 · Candidate matching

Hard constraints are scored **in code**. The LLM only explains. Every run records its model versions and its scoring version, so any number on screen can be traced back to the rules that produced it.

There are two entry points that share one scorer:

|           | CV CORPUS                               | PIPELINE                                          |
|-----------|-----------------------------------------|---------------------------------------------------|
| Source    | Uploaded CVs, parsed and embedded       | Apify-enriched LinkedIn profiles for one job      |
| Retrieval | Vector store, top MATCH_RETRIEVE_K = 50 | The selected candidates — small, scored in memory |
| Streaming | Batch                                   | Per candidate, live log, ranked as they land      |
| Photos    | None                                    | Yes                                               |



RETENTION

Runs persist **every candidate scored**, not just the top 5 — `analysis.candidates` . The top-N is a *view*. A recruiter can only judge whether a ranking is right by seeing what it rejected and why.

## Weights, and why they move

Four components. The nominal weights:

PROFILE FIT

0.50

MUST-HAVE SKILLS

0.30

EXPERIENCE

0.12

LOCATION

0.08

**A component the JD never stated is not scored as a free 100 — it is dropped, and its weight is redistributed over the components that do apply.** This is the rule

that makes scores mean something. A vague JD naming no location used to hand every candidate the same 8 points of padding, so only profile fit could discriminate, and everyone clustered at the same number.

| COMPONENT        | APPLIES WHEN     | NOMINAL | EFFECTIVE (PAYROLL JD) |
|------------------|------------------|---------|------------------------|
| Profile fit      | always           | 0.50    | 0.543                  |
| Must-have skills | the JD lists any | 0.30    | 0.326                  |
| Experience       | minYears is set  | 0.12    | 0.130                  |
| Location         | the JD names one | 0.08    | 0 – excluded           |

## How each component is valued

- **Profile fit** — cosine similarity between the JD embedding and the candidate's profile embedding ( `text-embedding-3-small` , 1536 dims). The only component that weighs the whole document rather than a checklist. Shown to recruiters as Strong / Good / Moderate / Weak.
- **Must-have skills** — sum of per-skill credit ÷ number of must-haves. See below.
- **Experience** — 1.0 when `totalYears ≥ minYears` ; 0.5 when the CV states no total; otherwise `years / minYears` .
- **Location** — 1.0 on a match, 0.6 when the candidate's location is unknown, 0.3 when clearly elsewhere.

## Skill matching rules

Every must-have is credited 0 – 1 , with the evidence recorded. **Every rule is evaluated and the strongest wins** — not first-match-wins, which let a weak rule cap a strong one.

| TIER     | CREDIT      | RULE                                                                                 | EXAMPLE                                                                     |
|----------|-------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| EXACT    | 1.00        | Normalised strings equal                                                             | Sozialversicherungsrecht                                                    |
| SPECIFIC | 1.00        | The requirement appears in the candidate's term, on token boundaries                 | role Personalsachbearbeiterin<br>Entgeltabrechnung COVERS Entgeltabrechnung |
| FUZZY    | 1.00 / 0.75 | <code>token_sort_ratio ≥ 95</code> / <code>≥ 88</code> , both <code>≥ 4</code> chars | SAP HR → SAP HR3 (92%)                                                      |
| BROADER  | 0.50        | The candidate's term is part of the requirement                                      | Lohnsteuer → Lohnsteuerrecht                                                |
| NONE     | 0.00        | Nothing evidences it                                                                 | —                                                                           |

## Two guards that took real bugs to find

### THE “R” BUG

The original matcher used a bare substring test in both directions. The one-letter skill “R” is a substring of *arbeitsrecht*, *sap hr3*, *entgeltabrechnung* — **every** German compound. Every data engineer scored 100% must-have coverage on an SAP payroll role. Fixed by requiring **token boundaries**.

### GERMAN COMPOUNDS

But token boundaries are too strict for German: *Lohnsteuer* sits inside *Lohnsteuerrecht* as a prefix, not a token. A **minimum-length floor of 5 characters** separates the two — long enough that the overlap can't be coincidence, which credits *Lohnsteuer* while still blocking “R”.

`token_sort_ratio` is used rather than `token_set_ratio`: the latter returns 100 whenever one token set is a subset of the other, which is exactly the over-crediting the containment rules above handle deliberately.

## The evidence pool

**Skills are not the only evidence a candidate has a skill.** Scoring `profile.skills` alone reported *Entgeltabrechnung* as *missing* for a woman whose job title is *Personalsachbearbeiterin Entgeltabrechnung* — the requirement is literally her job.

```
evidence pool = skills          ← first, so a real skills hit is named as the source
                  + currentTitle
                  + titles[]
                  + experience[].title
```

Order matters: skills come first so an exact skills match wins the evidence race and is quoted as the source, rather than the title.

## The must-have ceiling

Prose cannot buy a candidate past the bar their missing must-haves set. Coverage caps the final score:

| COVERAGE | CEILING      |
|----------|--------------|
| 100%     | 100 – no cap |
| ≥ 75%    | 85           |
| ≥ 50%    | 65           |
| ≥ 25%    | 50           |
| > 0%     | 40           |
| 0%       | 25           |

Because a ceiling flattens many candidates onto the same displayed number, ranking **tie-breaks on the uncapped base score** — the ordering underneath still carries signal.

## A worked example

Sina Baumann, *Personalsachbearbeiterin Entgeltabrechnung*, against the German payroll role.

| REQUIREMENT              | CREDIT | VIA                              | RULE            |
|--------------------------|--------|----------------------------------|-----------------|
| Entgeltabrechnung        | 1.00   | her job title                    | <b>SPECIFIC</b> |
| Arbeitsrecht             | 1.00   | “Arbeitsrecht (Grundkenntnisse)” | <b>SPECIFIC</b> |
| Sozialversicherungsrecht | 1.00   | listed directly                  | <b>EXACT</b>    |
| SAP HR3                  | 0.75   | “SAP HR” — 92% similar           | <b>FUZZY</b>    |
| Lohnsteuerrecht          | 0.50   | “Lohnsteuer”                     | <b>BROADER</b>  |

coverage =  $(1.00 + 1.00 + 1.00 + 0.75 + 0.50) / 5 = 4.25 / 5 = 85\%$

profile fit  $71.5 \times 0.543 = 38.8$

must-haves  $85.0 \times 0.326 = 27.7$

experience  $100.0 \times 0.130 = 13.0$  (3.8 years vs 3 required)

location excluded (JD names none – weight redistributed)

base 79.6

ceiling (85% coverage) = 85 → not binding

FINAL 79.6 Strong match

For contrast, before the title counted as evidence and before the compound rule, the same candidate scored **65.0** with 55% coverage — capped, and reported as *missing* the very skill she is employed to do.

## Gaps vs partials — and what the LLM is allowed to say

A **gap** is an absence: nothing in the profile evidences it. A **partial** is real evidence that falls short. Conflating them — treating everything under full credit as a gap — produced a screen that said *"Missing specific experience with SAP HR3"* directly above evidence crediting *SAP HR* at 92%.

```
gaps    = [ credit == 0 ]      → "Missing"
partial = [ 0 < credit < 1 ]  → "Partially met"
full    = [ credit >= 1 ]     → counted, not shown as a problem
```

invariant: every must-have is exactly one of the three – never both, never neither.

### THE STRUCTURAL FIX

The reasoning model used to be handed the (wrong) gap list as `missingMustHave`, and its prose then *overrode* the deterministic data. It wasn't reasoning from the profile — it was rationalising a bad input. Now the scorer is authoritative and **the model narrates in reasons only**. It cannot restate a fact, so it cannot contradict one.

# All prompts

The LLM is used in exactly five narrow places. Each is `temperature = 0`, strict JSON, defensive parse, with retries.

## 1 • PARSE JD → REQUIREMENTS

gpt-4o-mini

### SYSTEM

You extract structured hiring requirements from a job description. Return ONLY a JSON object, no commentary. Use null/empty when unknown.

### USER

From this job description, extract JSON with EXACTLY these keys:

```
{
  "title": string|null,
  "mustHaveSkills": string[],
  "niceToHaveSkills": string[],
  "minYears": number|null,
  "location": string|null,
  "seniority": string|null,
  "responsibilities": string[]
}
```

Job description:

<first 12,000 chars>

## 2 • EXTRACT CV → PROFILE

gpt-4o-mini

### SYSTEM

You extract structured data from a candidate CV. Return ONLY a JSON object, no commentary. Use null/empty when unknown. Do not invent skills.

### USER

From this CV, extract JSON with EXACTLY these keys:

```
{
  "fullName": string|null,
  "email": string|null,
  "phone": string|null,
  "linkedin": string|null,
  "location": string|null,
  "totalYears": number|null, // best estimate of total years of experience
  "currentTitle": string|null,
  "skills": string[], // normalized skill names
  "titles": string[], // past job titles
  "education": string[],
  "certifications": string[],
  "experience": [{"company": string|null, "title": string|null,
    "start": string|null, "end": string|null, "summary": string|null}]
}
```

CV:

<first 12,000 chars>

**SYSTEM**

You are a recruiting assistant. For each candidate, explain in plain language why they fit (or don't) the role, grounded **ONLY** in the skills/experience given. Never reference name, gender, age, or contact details. Return **ONLY** JSON.

**USER**

Role requirements:

```
{...the Role Spec requirements...}
```

Candidates (anonymized):

```
[{ "id", "currentTitle", "totalYears", "skills",
  "missingMustHave": [...],          ← credit == 0, authoritative
  "partialMustHave": [{skill, credit, via, method, note}] }]
```

Return JSON: {

```
  "candidates": [
    {"id": string, "reasons": string[2-3], "gaps": string[0-2]}
  ]
}
```

`reasons` = concrete, evidence-based bullets on why to call this person.

`gaps` = **ONLY** the requirements in `missingMustHave`. That list is

authoritative: it is the deterministic scorer's finding that **NOTHING** in the profile evidences those skills.

`partialMustHave` is **NOT** a gap and must **NEVER** be described as missing, lacking or absent. Each entry names real evidence the candidate has that partly satisfies the requirement (e.g. they list 'SAP HR' where the role wants 'SAP HR3', or 'Lohnsteuer' where it wants 'Lohnsteuerrecht'). If you mention one, describe it accurately as partial or adjacent – say what they **DO** have and how it falls short, never that they lack it. Writing 'missing X' for a partial match contradicts the evidence shown to the recruiter alongside your text.

Do not invent gaps that are in neither list.

**BIAS CONTROL**

The reasoning prompt is given **only skills and experience** — never name, contact details or demographics.

Candidates are passed anonymised, by id.

Prompts **4 • Strategist** and **5 • Broadener** are reproduced in full in [the Strategist](#) and [the Broadener](#) above.

# Config & cost

| SETTING                       | DEFAULT                            | WHAT IT DOES                               |
|-------------------------------|------------------------------------|--------------------------------------------|
| EXTRACTION_MODEL              | gpt-4o-mini                        | CV + JD structured extraction              |
| REASONING_MODEL               | gpt-4o-mini                        | Top-N candidate reasoning                  |
| EMBEDDING_MODEL               | text-embedding-3-small             | 1536 dims, JD + profiles                   |
| SOURCING_STRATEGY_MODEL       | openai:gpt-4o                      | Strategist — one call per prefill          |
| SOURCING_BROADEN_MODEL        | openai:gpt-4o-mini                 | Broadener — one call per zero-result retry |
| SOURCING_MAX_BROADEN_ATTEMPTS | 3                                  | Hard cost guard on retries                 |
| APIFY_SEARCH_ACTOR            | harvestapi/linkedin-profile-search | Short mode ≈ \$0.10 / page                 |
| PRESCREEN_ENABLED             | true                               | Gate hits before enrichment spend          |
| PRESCREEN_MIN_SCORE           | 25                                 | Drop below this — calibrated, not guessed  |
| MATCH_RETRIEVE_K              | 50                                 | Candidates pulled from the vector store    |
| MATCH_REASON_TOP_N            | 10                                 | How many get LLM prose                     |
| MATCH_RETURN_TOP              | 5                                  | The default view — not what is stored      |

Cost is bounded in three places: the broaden cap, the Broadener refusing to repeat a filter set it already tried, and the gate withholding enrichment from off-role hits.

# Known traps

Live findings, kept here because each one cost real time to diagnose.

## QUOTA LOOKS LIKE “NO CANDIDATES”

The Apify actor reports an account refusal as `Status: SUCCEEDED`, `Message: free user run limit reached` with an *empty dataset*. Read naively, that is indistinguishable from “nobody matches your search” — so the recruiter is told their role has no candidates, and the Broadener burns its entire budget widening searches that never ran.

`_raise_if_quota_exhausted` now inspects `statusMessage` and raises instead.

**Rule:** before blaming filters or titles for a zero-result search, replay a known-good filter set and prove the actor still returns anything. A bare `{searchQuery: "Payroll", locations: ["Germany"]}` returning 0 is never a real result.

## STILL OPEN

- `profileLanguages` **never reaches the actor** — the log shows `dropping unknown profileLanguages value 'de'`. The German-language filter has never done anything.
- `seniorityLevel` is not in the actor's input either. Its real inputs are only `search`, `locations`, `currentJobTitles`, `yearsOfExperienceIds`, `functionIds`.
- `function` is an unstable guess — `1` (Accounting) on one run and `10` (Finance) on the next for the same JD, when payroll people tag themselves `12` (Human Resources).
- `yearsOfExperience` is mis-mapped from `minYears : a minYears: 3` JD produced code `4` (= 6–10 years). Should be mapped deterministically, not by an LLM.
- The Broadener invents locations the actor rejects — `"Europe"` → 404.
- A discovery process that dies leaves `searchStatus: "running"` forever; there is no stale-claim timeout, so the job can never be re-run.

## READING A RUN

Read the **Apify actor log**, never a `searchAttempts` snapshot. The snapshot records what we *intended* to send, and a mid-flight read shows fewer attempts than actually ran.